PR #215: docs(getting-started): replace Deploy LLMs with Deploy Models built on the tt CLI and TT-Studio

https://github.com/tenstorrent/tenstorrent.github.io/pull/215
Branch `deploy-models-tt-cli` → `getting-started-tt-cli` · author ndalyTT · head `a2a00eeb`

Rendered review copy of the pages this PR changes, built with Sphinx from the PR head on top of its base branch (local merge commit `946c401`), generated 2026-10-26. Pages render in the site theme with the navigation removed; code blocks wrap instead of scrolling.

Commits (1)

`a2a00eeb`	2026-10-23	docs(getting-started): replace Deploy LLMs with Deploy Models built o…

Files changed (9)

`core/_extra/AGENTS.md`	+1	-1
`core/_extra/llms.txt`	+2	-2
`core/_templates/home.html`	+3	-3
`core/conf.py`	+2	-0
`core/getting-started/README.md`	+1	-1
`core/getting-started/deploy-models.md`	+468	-0
`core/getting-started/tt-software-stack.md`	+1	-1
`core/getting-started/vLLM-servers.md`	+0	-214
`core/index.rst`	+1	-1

Pages in this PDF

1. `getting-started/deploy-models.html`
2. `index.html`
3. `getting-started/README.html`
4. `getting-started/tt-software-stack.html`

Deploy Models

This page shows how to deploy a model on your Tenstorrent system and start sending it requests. You have two surfaces to work from, and both draw on the same catalog of models that Tenstorrent validates on each hardware generation:

- **The `tt` CLI.** `tt serve <model>` gives you an OpenAI-compatible API endpoint in one command. This is the path for applications, scripts, agents, and anything else that talks to an API.
- **TT-Studio.** A web application that deploys the same models and gives you a browser interface for them: chat, image and video generation, speech-to-text, object detection, and more. This is the path when you want to try a model from a browser, or hand it to people who do not use a terminal.

The flow is the same whatever kind of model you deploy. The catalog is not limited to large language models: it includes diffusion models for image and video generation, speech and audio models, decision models, and tabular foundation models, and `tt serve` and TT-Studio treat each of them the same way. Under the hood both run TT-Inference-Server, the serving engine that packages every model with the right runtime in a container. You do not need to clone or configure it yourself.

The Same Steps for Every Model

Deploying any model in the catalog takes the same five steps, and each is one command:

Step	Command	What it does
1. See what runs on your system	`tt model list`	Lists the models validated for th
2. Check what the model needs	`tt model info <model>`	Reports the disk and memory th
3. Deploy it	`tt serve <model>`, or `tt serve <model> --studio` for TT-Studio	Downloads the weights and the
4. Send a request	`curl http://localhost:20000/v1/<route>`	Every server speaks the OpenAI
5. Manage and stop	`tt model ps`, `tt model logs <model>`, `tt model stop <model>`	Shows what is running, streams

The commands do not change from one model to the next. What changes is what they report: the resources a model needs, the stages `tt serve` passes through while it loads, the route you call, and what comes back. The rest of this page walks the steps with `Wan2.2-T2V-A14B-Diffusers`, the Wan 2.2 text-to-video diffusion model, as the example, and says where another kind of model behaves differently. The example needs four chips, so it runs on a TT-QuietBox 2, a TT-LoudBox, or a Galaxy; on a single-chip board, follow the same steps with a model from your own list. Installing Tenstorrent Software walks the same steps with an LLM.

Before You Begin

Important

- This guide assumes you have completed Installing Tenstorrent Software: `tt update` has run, `tt device status` lists your devices, and Docker runs without `sudo`.
- You are logged in to Hugging Face with `tt model login`. Most models download their weights from the Hub, and the login is needed even for models that have no license gate.
- Disk space depends on the model. `tt model info <model>` reports the minimum for each one, from 15 GB for a speech or image model to 160 GB for a 70B-parameter LLM. Downloaded weights are cached under `~/.cache/huggingface` and shared by every path that serves the model.

(Wormhole™ only)

❗ If you are using a Wormhole TT-QuietBox® or TT-LoudBox™ you must complete the following step

For these systems you must configure a system-level mesh topology between the Wormhole™ Networked AI Processors. Run the following script to install `tt-topology` and configure the mesh.

```
TMP_DIR=$(mktemp -d); (trap 'echo "---"; echo "Cleaning up..."; if type deactivate &>/dev/null; then deactivate; fi; echo "Removing temporary directory: $TMP_DIR"; rm -rf "$TMP_DIR"; cd; echo "Cleanup complete."' EXIT; trap 'echo -e "\033[0;31m!!! ERROR: Failed to configure mesh topology\033[0m"' ERR; set -e; cd "$TMP_DIR"; echo "Working in temporary directory: $TMP_DIR"; echo "---"; echo "Creating Python virtual environment..."; python3 -m venv tt-topology-venv; source tt-topology-venv/bin/activate; echo "Virtual environment activated."; echo "---"; echo "Installing tt-topology from git..."; pip install --quiet git+https://github.com/tenstorrent/tt-topology.git; echo "tt-topology installed."; echo "---"; echo "Running tt-topology command. This may take a moment..."; tt-topology -l mesh; echo "---"; echo "Script finished successfully.";)
```

Step 1: Choose a Model

1. See what runs on your system

`tt model list` shows every model that is validated for the hardware it detects, from two sources: the released catalog that Tenstorrent ships and tests, and community bundles that anyone has published for Tenstorrent hardware.

```
tt model list
```

Each row names the model, its `source` (`tt-inference-server` or `tt-studio` for the catalog, `HuggingFace` for a community bundle), the `engine` that runs it (`vLLM` for language models, `media` for image, video, audio, and embedding models, and `forge` for the vision and embedding models that run through TT-Forge), the smallest board or mesh each capability needs, and whether the weights are already on disk. Narrow the list to one kind of model with `--type`:

Kind of model	`--type`	Examples in the catalog
Large language models	`llm`	`Qwen3.8-27B`, `Llama-3.3-70B-Instruct`, `gpt-oss-120b`
Vision-language models	`vlm`	`Qwen2.5-VL-7B-Instruct`, `Llama-3.2-11B-Vision-Instruct`
Image generation (diffusion)	`image`	`FLUX.1-schnell`, `FLUX.1-dev`, `stable-diffusion-xl-base-1.0`
Video generation (diffusion)	`video`	`Wan2.2-T2V-A14B-Diffusers`, `mochi-1-preview`
Speech recognition	`audio`	`whisper-large-v3`, `distil-large-v3`
Text to speech	`text_to_speech`	`speecht5_tts`
Vision classification and detection	`cnn`	`yolox_nano`, `resnet-50`, `segformer`
Embeddings	`embedding`	`bge-m3`, `Qwen3-Embedding-4B`

The example model on this page is a video model. Listing that kind on a TT-QuietBox 2 gives a list of one:

```
tt model list --type video
```

```
Models (tt catalog + community) for p300x2 (detected — `tt model list --all` for every
                                device/bundle)
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━┳━━━━━━━━━━━━━━━━┳━━━━━━━━━┓
┃ name                      ┃ source              ┃ engine ┃ serving profiles ┃ weights ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━╇━━━━━━━━━━━━━━━━╇━━━━━━━━━┩
│ Wan2.2-T2V-A14B-Diffusers │ tt-inference-server │ media  │ p150x4, p300x2   │ —       │
└───────────────────────────┴─────────────────────┴────────┴──────────────────┴─────────┘
```

A model that is known not to work on your board is left out. `tt model list --all` shows every model on every device, and `--hw p150x4` shows the list for another configuration without touching the hardware. `--catalog` and `--community` show one source at a time.

2. Check what a model needs

Before you commit to a download, inspect the model. `tt model info <model>` reports where the weights come from, which surfaces can deploy the model, what it needs from the host, and how it is validated on each hardware configuration. For the example model:

```
tt model info Wan2.2-T2V-A14B-Diffusers
```

```
                                    Wan2.2-T2V-A14B-Diffusers
┌──────────────────────┬──────────────────────────────────────────────────────────────────────────────────────────┐
│ hf_repo              │ Wan-AI/Wan2.2-T2V-A14B-Diffusers                                                         │
│ type                 │ video                                                                                    │
│ engines              │ media                                                                                    │
│ via                  │ inference-server, studio                                                                 │
│ servable             │ yes — `tt serve Wan2.2-T2V-A14B-Diffusers` (`--studio` deploys it with TT-Studio)        │
│ parameters           │ 14B                                                                                      │
│ min disk             │ 60 GB                                                                                    │
│ min ram              │ 32 GB                                                                                    │
│ cached               │ no                                                                                       │
│ on t3k               │ media — COMPLETE, max context 65536                                                      │
│ on galaxy            │ media — COMPLETE, max context 65536                                                      │
│ on p150x4            │ media — COMPLETE, max context 65536                                                      │
│ on p150x8            │ media — COMPLETE, max context 65536                                                      │
│ on p300x2            │ media — COMPLETE, max context 65536                                                      │
│ on blackhole_galaxy  │ media — COMPLETE, max context 65536                                                      │
└──────────────────────┴──────────────────────────────────────────────────────────────────────────────────────────┘
```

The rows that matter when you choose:

- `via` lists the surfaces that can deploy the model. Most catalog models offer both `inference-server`, the path `tt serve` takes by default, and `studio`. A few are carried only by TT-Studio, and `tt serve` sends those to it automatically.
- `min disk` and `min ram` are what the model needs on the host, and `cached` says whether its weights are on disk already. The LLMs in the catalog range from 36 GB of disk and 20 GB of memory for an 8B-parameter model to 160 GB and 175 GB for `Llama-3.3-70B-Instruct`. A download can be larger than the minimum: the example model's weights come to 117.5 GB.
- `on <hardware>` gives the validation status for each configuration. `COMPLETE` is fully validated; `EXPERIMENTAL` runs but has not been through the full test suite. A configuration that is missing from the list is one the model is not validated on, which is why the example model shows no single-chip boards. A single-chip model on a larger system shows `served as p150` or the equivalent chip: it takes one chip and leaves the rest free.

3. Get access to gated models

Some model owners gate their weights behind a license. Meta's Llama models, Google's Gemma models, and Black Forest Labs' FLUX.1 models all require you to accept their terms on the model's Hub page before you can download them, and the owner may take days to grant access. Qwen, Wan, and Whisper models are not gated.

To get access, open the `hf_repo` page that `tt model info` shows, accept the license, and make sure the account you accepted with is the one you used for `tt model login`. Until access is granted, serving a gated model stops at the download:

```
  ✓ host ready  tt-inference-server 0.25.0
  ✗ downloading weights  black-forest-labs/FLUX.1-schnell
╭──────────────────────────── error [TOOL_FAILED] ─────────────────────────────╮
│ FLUX.1-schnell stopped before the server was ready.                          │
│ AssertionError: ⛔ Error during: … hf download black-forest-labs/FLUX.1-schnell │
╰──────────────────────────────────────────────────────────────────────────────╯
```

The full log it names ends with `Error: Access denied. Model 'black-forest-labs/FLUX.1-schnell' requires approval.`

Tip

`tt model pull <model>` downloads a model's weights ahead of time, so the serve itself is faster. It is also a quick way to confirm that your access to a gated model works.

Step 2: Deploy with the CLI

1. Preview, then serve

`tt serve <model> --dry-run` shows how the model would be deployed without starting anything: the serving path, the device configuration, the container image, where the weights go, and the port. For the example model:

```
tt serve Wan2.2-T2V-A14B-Diffusers --dry-run
```

```
✓ Detecting device configuration  p300x2 (override with --device)
                            tt serve Wan2.2-T2V-A14B-Diffusers — dry run
┌──────────────────┬──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ backend          │ tt-inference-server                                                                                  │
│ workflow         │ server                                                                                               │
│ device           │ p300x2                                                                                               │
│ engines          │ media                                                                                                │
│ status           │ COMPLETE                                                                                             │
│ docker image     │ ghcr.io/tenstorrent/tt-media-inference-server:0.17.0-8c48a10                                         │
│                  │ override — the spec pins                                                                             │
│                  │ ghcr.io/tenstorrent/tt-media-inference-server:0.17.0-8c48a10                                         │
│ tool-call parser │ —                                                                                                    │
│ reasoning parser │ —                                                                                                    │
│ tt config        │ {"trace_region_size": 30000000} (from the model spec)                                                │
│ weights cache    │ /home/ttuser/.cache/huggingface                                                                      │
│ HF token         │ from the Hugging Face login store                                                                    │
│ port             │ 20000  (server default)                                                                              │
└──────────────────┴──────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

`device` is the configuration the model is deployed on, here the whole system: the example model runs across all four chips of a TT-QuietBox 2. `weights cache` shows that the weights go to the host's Hugging Face cache, where every path that serves the model can reuse them, and `tt config` carries the settings the catalog pins for the model. Below the table, `tt serve` prints the exact command it would run.

`tt serve <model>` deploys the model. On first use it fetches the weights and pulls the container image; then it opens the devices, loads the model, and starts the server, showing each stage as a live checklist with its elapsed time. It returns once the endpoint actually answers requests. For the example model:

```
tt serve Wan2.2-T2V-A14B-Diffusers
```

```
  ✓ host ready  tt-inference-server 0.25.0
  ✓ weights ready  Wan-AI/Wan2.2-T2V-A14B-Diffusers  16m 58s
  ✓ image ready  tt-media-inference-server:0.17.0  2m 13s
  ✓ container started  tt-inference-server-2f018ce0  3.7s
  ✓ server started  4.5s
  ✓ Tenstorrent device opened  4 chips  4.3s
  ✓ model loaded  2/2  5m 52s
  ✓ endpoint answering
✓ Phase 3/3 · Start  25m 22s
┌─ Wan2.2-T2V-A14B-Diffusers ready ──────────────────────────────────────────┐
│                                                                            │
│  endpoint  http://127.0.0.1:20000/v1                                       │
│  models    curl http://127.0.0.1:20000/v1/models                           │
│  chat      tt launch                                                       │
│                                                                            │
│  Ready in 25m 22s · via tt-inference-server                                │
│  Logs · tt model logs Wan2.2-T2V-A14B-Diffusers --follow                   │
│  Stop · tt model stop Wan2.2-T2V-A14B-Diffusers                            │
│                                                                            │
└────────────────────────────────────────────────────────────────────────────┘
```

The stages are the same for every model; the times are not. Most of a first serve is the download: the example model's 117.5 GB of weights took 17 minutes here and the container image two, ahead of a six-minute load that also writes a cache for your hardware. Later serves skip the download and the pull; the same serve from a warm cache took six minutes on this system, and `tt model list` now shows the size of the weights in its `weights` column. An LLM's checklist has stages of its own,

for sizing the KV cache and warming the model up; Installing Tenstorrent Software shows one for `Qwen3.8-27B`. To watch the server's own log while you wait, run `tt model logs <model> --follow` in a second terminal. Pressing Ctrl-C in the `tt serve` terminal stops watching, not the server.

2. Send a request

The endpoint speaks the OpenAI API, so any OpenAI-compatible client can use it. Point the client at `http://localhost:20000/v1` with any value for the API key. Which route you call depends on the kind of model:

Kind of model	Route	What you send
Large language and vision-language models	`/v1/chat/completions`	a list of `messages`
Speech recognition	`/v1/audio/transcriptions`	the audio file, base64-encod
Text to speech	`/v1/audio/speech`	the `text` to speak
Image generation	`/v1/images/generations`	a `prompt`, with optional `num_`
Video generation	`/v1/videos/generations`	a `prompt`; the reply is a job th and download from `/downloa`
Embeddings	`/v1/embeddings`	the `input` text

For an LLM, `tt model curl` sends a chat completion without any JSON on your part, and `--print` shows the equivalent `curl` command for your own scripts:

```
tt model curl "In one sentence, what is Tenstorrent?" --max-tokens 2048
```

For the other kinds of model, call the route directly. With the example model, generate a short video. Video generation runs as a job: the request returns at once with the job's id and status, you poll the job until it is `completed`, and then you download the result.

```
curl -s http://localhost:20000/v1/videos/generations \
  -H "Content-Type: application/json" \
  -d '{"prompt": "A red fox trotting through fresh snow in a pine forest, soft morning light,
camera tracking alongside"}' | python3 -m json.tool
```

```
{
    "id": "696734f5-1990-4bb9-8197-51f172e8bad4",
    "job_type": "video",
    "status": "queued",
    "created_at": 1792683962,
    "model": "/home/container_app_user/readonly_weights_mount/Wan2.2-T2V-A14B-
Diffusers/snapshots/5be7df9619b54f4e2667b2755bc6a756675b5cd7",
    "request_parameters": {
        "prompt": "A red fox trotting through fresh snow in a pine forest, soft morning light,
camera tracking alongside",
        "negative_prompt": null,
        "num_inference_steps": 20,
        "seed": null
    }
}
```


The default is 20 denoising steps; `num_inference_steps`, `negative_prompt`, and `seed` are the optional fields. Check on the job with its `id`:

```
curl -s http://localhost:20000/v1/videos/generations/<job_id> | python3 -m json.tool
```


```
{
    "id": "696734f5-1990-4bb9-8197-51f172e8bad4",
    "job_type": "video",
    "status": "completed",
    "created_at": 1792683962,
    "model": "/home/container_app_user/readonly_weights_mount/Wan2.2-T2V-A14B-
Diffusers/snapshots/5be7df9619b54f4e2667b2755bc6a756675b5cd7",
    "request_parameters": {
        "prompt": "A red fox trotting through fresh snow in a pine forest, soft morning light,
camera tracking alongside",
        "negative_prompt": null,
        "num_inference_steps": 20,
        "seed": null
    },
    "completed_at": 1792684206
}
```


Once `status` is `completed`, download the clip:

```
curl http://localhost:20000/v1/videos/generations/<job_id>/download --output fox.mp4
```

On a TT-QuietBox 2 the job took four minutes and produced a five-second clip, 832 by 480 pixels at 16 frames per second.

3. Manage what is running

```
tt model ps              # what is being served, on which port, and whether it is healthy
tt model logs <model>    # the server's log (--follow to stream it)
tt model stop <model>    # stop the server; the weights stay cached
```

With the example model running, `tt model ps` shows:

```
                                                Served models
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━┳━━━━━━━━━┓
┃ name                                  ┃ backend          ┃ container                           ┃ port   ┃ health  ┃
┃ uptime                                ┃                  ┃                                     ┃        ┃         ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━╇━━━━━━━━━┩
│ Wan2.2-T2V-A14B-Diffusers             │ inference-server │ tt-inference-server-2f018ce0        │ 20000  │ healthy │
│ 6m                                    │                  │                                     │        │         │
└───────────────────────────────────────┴──────────────────┴─────────────────────────────────────┴────────┴─────────┘
```

`tt serve` takes `--port` to serve on a port other than 20000, and `--device` to override the detected configuration. Anything it does not recognize after `--` is passed through to the server itself.

Step 3: Deploy with TT-Studio

TT-Studio deploys the same catalog through a web application. It brings up its services in Docker, deploys the model you choose, and gives you an interface for it in the browser:

- **Chat** for language models, with streaming replies, a history of conversations, file and image upload, voice input, a panel that shows a reasoning model's thinking, and inference statistics such as time to first token.
- **Knowledge bases** for chat: upload documents, and TT-Studio indexes them so the model can answer questions about them.

- **Image generation** and **video generation** for diffusion models, with a gallery of generated results you can download.
- **Speech-to-text** for recognition models: record from the browser or upload a file, and keep the transcriptions.
- **Object detection** for vision models, from an uploaded image or a live webcam feed.
- **A voice agent** that chains a speech recognition model, a language model, and a text-to-speech model.

Every model TT-Studio deploys is also served on an API endpoint, so applications can use it at the same time as the browser does.

1. Start TT-Studio and deploy a model

`tt serve <model> --studio` deploys a model with TT-Studio instead of directly. TT-Studio takes port 3000 and the devices for itself, so stop anything `tt serve` or `tt launch` started first. A model server that was stopped by `tt model stop` leaves the devices in a state TT-Studio's deploy cannot initialize, so reset them before you start. To move the example model from the CLI to TT-Studio:

```
tt model stop Wan2.2-T2V-A14B-Diffusers
tt device reset
tt serve Wan2.2-T2V-A14B-Diffusers --studio
```

`tt device reset` asks you to confirm, because it interrupts anything running on the devices; answer `y`. On first use `tt` fetches the latest TT-Studio release. TT-Studio asks you to accept its terms and offers to add a `tt-studio` shortcut to your shell, which gives you its own commands later in this step. It then prints a `TT Studio is ready` box with its web UI at `http://localhost:3000`, deploys the model, and prints a ready box with the endpoint and the page that manages the deployment:

```
┌─ TT Studio is ready ──────────────────────────────────────────────────────┐
│                                                                           │
│  URL            ● http://localhost:3000                                   │
│  Mode           Local + TT Hardware                                       │
│  Hardware       QuietBox (QB2) · 4 device(s)                              │
│  Search Agent   Disabled · set TAVILY_API_KEY (app.tavily.com)            │
│                                                                           │
│  Ready in 36.3s · 5 phases                                                │
│  Stop · python run.py --stop                                              │
│  Logs · python run.py --logs                                              │
│  Info · python run.py --info                                              │
│  Help · python run.py --help                                              │
│                                                                           │
└───────────────────────────────────────────────────────────────────────────┘

[15:52:42] resolved model_id: id_tt-metal-Wan2.2-T2V-A14B-Diffusers-v0.25.0 (model_name ==
'Wan2.2-T2V-A14B-Diffusers')

🚀 Deploying Wan2.2-T2V-A14B-Diffusers (auto-allocated slot)
  Ctrl-C stops watching; the deploy keeps running in the backend.
  ✓ Starting container 3s
  Deployed ━━━━━━━━━━━━━━━━━━━━━━━━ Deployment completed successfully
  Waiting for the model to answer health checks…

┌─ Wan2.2-T2V-A14B-Diffusers is ready ──────────────────────────────────────┐
│                                                                           │
│  Model      Wan2.2-T2V-A14B-Diffusers                                     │
│  Chips      0, 1, 2, 3                                                    │
│  Endpoint   ● http://localhost:20000/v1/videos/generations                │
│  Health     http://localhost:20000/health                                 │
│  Web UI     http://localhost:3000/models-deployed                         │
│                                                                           │
│  python run.py --stop to stop · python run.py --logs for logs             │
│                                                                           │
└───────────────────────────────────────────────────────────────────────────┘
```

TT-Studio's own box gives you its web UI; the model's box gives the endpoint for its kind of model, here `/v1/videos/generations`, its health check, and the **Models Deployed** page. With the weights already on disk from Step 2, the deploy took about four minutes on a TT-QuietBox 2.

Without a model name, `tt serve --studio` lists everything TT-Studio can deploy on your system and asks which one you want. TT-Studio requires Docker.

2. Deploy from the browser instead

You can also start TT-Studio and let the browser drive the deployment. Open `http://localhost:3000`, go to the models page, and pick a model from the list, which is filtered to what your hardware supports. On a multi-chip system TT-Studio chooses the chips for you, and lets you pick them yourself when other models are already deployed. Deploy, and the page shows the progress of the download and container start, with the deployment log a click away. The **Models Deployed** page lists every running model with its health, the chips it occupies, its log, and a control to delete it.

3. Use the model

Once the model is healthy, the Models Deployed page links to the interface for its kind: chat for a language model, the image or video generator for a diffusion model, speech-to-text for a recognition model, and object detection for a vision model. The endpoint in the ready box, on port 20000, serves API clients at the same time; `tt model curl` and `tt launch` work against it the same way as against a model started by `tt serve`.

4. Stop

`tt model stop <model>` stops a model that TT-Studio deployed, resets the chips it was using, and takes TT-Studio's services down with it. Your settings, downloaded weights, chat history, and knowledge bases are kept for next time. With the `tt-studio` shortcut you can also manage TT-Studio directly:

```
tt-studio --stop-model <model>   # stop one model and keep TT-Studio running
tt-studio --status               # a live view of the running stack
tt-studio --logs                 # stream the logs of every TT-Studio service
tt-studio --stop                 # stop TT-Studio, keeping its data
```

Deploy a Community Model

Community bundles are models that anyone has packaged for Tenstorrent hardware and published on the Hugging Face Hub. `tt model list` includes the ones Tenstorrent has reviewed; `--include-unverified` shows the rest. They are served through the same commands, with the bundle's repository id in place of a catalog name:

```
tt model list --community                # bundles that fit your hardware
tt model info <namespace>/<name>         # the bundle's manifest and a compatibility verdict
tt serve <namespace>/<name>              # served through tt-model-manager
```

`tt` checks that the bundle was built for your chip architecture and that your system has enough chips before it serves, and asks for confirmation before it serves a bundle that has not been verified. Tenstorrent does not test or maintain community bundles. If you have brought a model up on Tenstorrent hardware yourself, the tt-model-manager documentation explains how to package and publish it so that anyone can serve it the same way.

Troubleshooting

The model I want is not in `tt model list`

The list is filtered to your detected hardware. `tt model list --all` shows every model on every device, and `tt model info <model>` says why one is left out for yours. A model that is validated on a smaller configuration than you have, such as a single-chip model on a TT-QuietBox 2, is listed and runs on part of the system.

The serve stops at `downloading weights` with "requires approval"

The model is gated and your Hugging Face account has not been granted access to it yet. Accept the license on the model's Hub page, wait for the owner to approve the request if the model requires that, and serve again. Make sure `tt model login` is for the same account.

`tt serve` fails with "Access to model … is restricted"

The model is gated and your account has not accepted its license yet, or the login on this machine is for a different account. Accept the license on the model's Hub page, check `tt model login`, and serve again. `tt model logs <model>` shows the server's own output for any failed start.

A serve or a TT-Studio deploy fails with "Try resetting the board"

A model server that was stopped or crashed can leave the devices in a state the next server cannot use. Reset every device, confirming with `y` when asked, and serve again:

```
tt device reset
```

`tt serve` says the model did not report ready within 60 minutes

`tt serve` waits up to an hour for the endpoint to answer, then stops watching and leaves the server running. A cold start of a large diffusion model compiles for your hardware and can be slow, so first check whether it is still making progress: `tt model logs <model> --follow` shows the server's output, and `tt model ps` reports it `healthy` once it answers. If the log shows an error instead, stop the server with `tt model stop <model>`. To wait longer next time, set `TT_SERVE_READY_TIMEOUT` to a number of seconds before you serve.

TT-Studio does not start because port 3000 is in use

TT-Studio needs port 3000 and does not move to another one. Stop whatever is using it first, for example a web UI started with `tt launch`: `tt launch stop openwebui`.

I am running out of disk space

`tt model rm <model>` removes a model's local artifacts and keeps its weights; add `--include-weights` to remove those too. The weights live under `~/.cache/huggingface`.

What to Do Next

Now that your model is running:

- **Connect an application to it** with `tt launch`. Open WebUI and AnythingLLM give you a chat interface in the browser; OpenCode, Aider, pi, Qwen Code, and Hermes Agent turn a tool-calling LLM into a coding agent in your terminal. `tt launch list` shows which are available on your machine, and Installing Tenstorrent Software walks through connecting one.
- **Watch it in real time** — run `tt-toplike` in a second terminal window. TT-Toplike shows live core activity, power draw, and memory traffic on the chips while they serve requests.
- **Use it as a coding assistant** — the Coding Assistant with Aider lesson walks through pairing a local endpoint with Aider for on-device pair programming.
- **Build agents on top of it** — if you have a TT-QuietBox® 2, the Local AI Agents on QuietBox 2 lesson covers tool-calling pipelines and multi-agent workflows against your local endpoint.

- **Generate images and video at scale** — if you have a TT-QuietBox® 2, TT-Local-Generator runs alongside inference with its own queue and gallery.
- **Check supported models** — the tt-inference-server repository is the authoritative source for which models are validated on each hardware generation, and the catalog `tt model list` shows is built from it.

Need Additional Support?

If you encounter any issues, or have a question that isn't covered in the documentation, please raise a support request. Our team will review your request and provide assistance.

Tenstorrent Documentation

No hardware yet? Get access to Tenstorrent silicon right in your browser with Tenstorrent Cloud Console

GET STARTED

Installing Tenstorrent Software

Set up drivers, firmware, and the TT-Metalium environment on your Tenstorrent hardware, then verify your install.

TT-VSCODE-TOOLKIT

Learn by Building

Get hands-on with a guided learning path — from making your first inference to building full applications with LLMs.

Popular Destinations

TT-Studio

Point-and-click web UI for deploying and interacting with models.

TT-Inference-Server

Deploy and serve models — the canonical source for validated model support.

TT-Metalium

Low-level C++ SDK for writing custom kernels on Tensix cores.

Get Started →

Overview

How TT-Forge, TT-NN, and TT-Metalium fit together — and which to use when.

Installing Tenstorrent Software

Install drivers, firmware, and the TT-Metalium environment, then verify your setup.

Manual Installation

Manually install system dependencies, drivers, and the software stack step by step.

Deploy Models

Serve LLMs, diffusion, speech, and vision models with the tt CLI or TT-Studio, and get an OpenAI-compatible endpoint.

Learn by Building

Get hands-on with guided, end-to-end lessons in the TT-VSCode-Toolkit.

Software →

TT-Forge™

End-to-end MLIR compiler. Compile models from PyTorch, JAX, and ONNX.

TT-XLA

PJRT-based frontend for running JAX and OpenXLA models on Tenstorrent hardware.

TT-NN™

High-level Python API of pre-optimized neural-network operations.

TT-Lang™

Python DSL for authoring fused custom operations.

TT-MLIR™

MLIR-based backend compiler infrastructure that lowers models onto Tenstorrent hardware.

TT-Metalium™

Low-level C++ SDK for writing custom kernels on Tensix cores.

Cloud-Native Support

Run Tenstorrent accelerators on Kubernetes — install, configure, and operate the device stack.

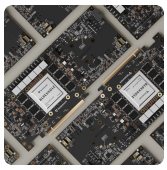
Hardware →

BLACKHOLE



TT-QuietBox® 2 (Blackhole)

Next-generation liquid-cooled Blackhole workstation.



Blackhole® PCIe Cards

Infinitely scalable, high-performance AI processing.



TT-QuietBox® (Blackhole)

Dead-quiet, liquid-cooled desktop AI.



Tenstorrent Galaxy® (Blackhole)

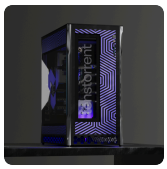
Scalable, enterprise-class AI server.

WORMHOLE



Wormhole™ PCIe Cards

Flexible, scalable processors built with Tensix Cores™.



TT-QuietBox® (Wormhole)

Dead-quiet desktop AI.



TT-LoudBox™ (Wormhole)

High-throughput workstation AI.



Tenstorrent Galaxy® (Wormhole)

Scalable, enterprise-class AI server.

Learn End-to-End through TT-VSCode-Toolkit →

Your First Inference

Set up the toolkit, detect hardware, and run your first model end-to-end.

Serving & APIs

Production inference with tt-inference-server and vLLM, plus image and video generation.

Applications

Real-world apps: coding assistants, AI agents, and video generation on QuietBox 2.

Custom Training

Datasets, fine-tuning, multi-device training, and training from scratch.

Cookbook

Hands-on projects: Game of Life, audio processing, fractals, and particle sims.

Compilers & Frameworks

Image classification with TT-Forge, JAX and PyTorch/XLA, and an intro to TT-Lang.

CS Fundamentals

RISC-V architecture, memory hierarchy, parallelism, synchronization, and complexity.

Advanced

Model bring-up bounties and exploring TT-Metalium at the kernel level.

Deployment

Deploy the toolkit and your work to the cloud with Koyeb.

Tools →

TT-Inference-Server

Deploy and serve models — the canonical source for validated model support.

TT-Studio

Point-and-click web UI for deploying a model — the fastest path for new users.

TT-SMI

Command-line device telemetry and firmware info — ships with the software stack.

TT-Toplike

Real-time terminal monitor: power, temperature, current, and core activity.

TT-NN Visualizer

Inspect model execution: graphs, memory plots, tensor and buffer views.

TT-Topology

Configure Ethernet routing on multi-card QuietBox and LoudBox systems.

TT-VSCode-Toolkit

Guided VS Code learning environment — browse tracks in the Learn section above.

TT-Blacksmith

Optimized ML training recipes for Tenstorrent hardware.

Explore the Ecosystem →

TT-Awesome

A curated directory of 115+ community and official projects, models, kernels, and research across 12 categories.

🪐 Planet Tenstorrent

The latest across the ecosystem — releases, papers, talks, videos, and demos, aggregated in one feed.

Tenstorrent on GitHub

All public source repositories — the software stack, compilers, tools, and demos.

Resources →



Request Support

Submit a support ticket and our team will help.



FAQ

Learn more about our hardware and software.



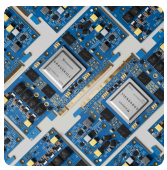
Contact Us

Get in touch with the Tenstorrent team.

Join Our Discord

Join the Tenstorrent developer community.

Legacy



Grayskull™

Tenstorrent's first-generation Tensix processor.



T7000

Earlier-generation Tenstorrent system.



T1000

Earlier-generation Tenstorrent system.

Installing Tenstorrent Software

This guide takes you from a fresh Tenstorrent system to a model generating tokens. You will install the Tenstorrent CLI, `tt`, use it to update your system to the most recent tested set of Tenstorrent software, confirm your hardware is visible, see which models your hardware can run, and serve a model.

`tt` is the single entry point to the Tenstorrent software stack. All layers of the stack can be accessed via the CLI. When you are ready to go deeper, for example to write kernels with TT-Metalium™, run models with TT-NN™, or deploy through a web interface with TT-Studio, `tt` is where you branch off into those projects.

Before You Begin

Note

Setting up your new TT-QuietBox® 2? Follow the Setup Guide.

Before you begin, ensure your host machine meets the following prerequisites:

- The host machine has an internet connection. The CLI downloads packages from PyPI and GitHub, the Tenstorrent package repositories, and model weights from the Hugging Face Hub.
- The host machine has a supported operating system installed (see below).
- You have administrator (`sudo`) privileges on the host machine. The driver, firmware tooling, and HugePages configuration are installed as root.
- The system is physically unboxed and set up according to your product's installation manual and safety guidelines.

Supported Operating Systems

Tenstorrent recommends Ubuntu LTS for all Tenstorrent software.

Operating system	Status
Ubuntu 22.04 LTS, Ubuntu 24.04 LTS	Supported
Debian 13, Fedora 42, Fedora 43	Supported, with less testing than Ubuntu
Other Debian and RPM based distributions	Untested and considered experimental
macOS, Windows, WSL	Not supported. There is no driver, so nothing hardware-relate

BIOS Requirements

The BIOS for your host motherboard should configure the PCIe AER Reporting Mechanism to be set to `OS First`. Tenstorrent's TT-SMI software requires this setting to function properly.

Important

If you are using a TT-QuietBox, you do not have to worry about setting this option. It is set to `OS First` by default.

If you update or reset your BIOS for any reason, you must reconfigure the PCIe AER Reporting Mechanism setting to `OS First` to ensure TT-SMI functions correctly. This setting is typically located in the **Motherboard Information** section of your BIOS.

Install the CLI Dependencies

The `tt` CLI is a Python application. It needs Python 3.10 or newer with the `venv` module, plus `git` and `curl`. Install them with your system package manager.

On Ubuntu or Debian:

```
sudo apt update && sudo apt install -y python3 python3-venv python3-pip git curl ca-
certificates
```

On Fedora:

```
sudo dnf install -y python3 python3-pip git curl
```

The CLI must live in its own isolated Python environment so that it can safely update itself later. Tenstorrent recommends uv to manage that environment. Install it with:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Then open a new terminal, or run `source ~/.bashrc`, so that `uv` is on your `PATH`.

Note

If you already use pipx, it works too. Install the CLI with `pipx install tenstorrent` in the next step instead of the `uv` command. Other installation options are described in the tt-cli developer notes.

Step 1: Install the Tenstorrent CLI

Install the CLI as a `uv` tool and make sure it is on your `PATH`:

```
uv tool install tenstorrent
uv tool update-shell
```

Open a new terminal, then confirm the CLI runs:

```
tt --version
```

The first time you run a `tt` command in a terminal it asks once whether you want to share anonymous usage telemetry. Telemetry is opt-in. Nothing is collected unless you say yes, and you can change your answer at any time with `tt config set telemetry.enabled true` or `false`. See the telemetry policy for exactly what is collected.

Run `tt --help` to see the command groups. The ones used in this guide are `update`, `device`, `model`, `serve`, and `launch`.

Note

`tt` is beta software and is evolving quickly. If you hit a problem, `tt report issue` opens a prefilled GitHub issue with your environment details already collected.

Step 2: Install the Software Stack with `tt update`

Tenstorrent publishes a tested set of component versions, called the **golden** versions, that are validated to work together: the kernel-mode driver (TT-KMD), device firmware, HugePages configuration, TT-SMI, TT-Flash, and a container runtime for serving models. `tt update` brings your system onto that set in one command. On a fresh machine it installs everything. On a machine that already has Tenstorrent software it changes only what differs from golden, so you can rerun it any time to get back to a known-good state.

To see what would change without changing anything, preview it first:

```
tt update --dry-run
```

The plan lists each component, the version installed, the golden version, and the action `tt update` would take:

```
✓ Fetching golden versions (v3.0.0)  v3.0.0
                      Update plan (goldens: golden.json v3.0.0)
┌──────────────────────┬──────────┬───────────┬──────────────────────────┬──────────┐
│ component            │ kind     │ installed │ golden                   │ action   │
├──────────────────────┼──────────┼───────────┼──────────────────────────┼──────────┤
│ tt-flash             │ uv-tool  │ —         │ 4.0.0                    │ install  │
│ tt-inference-server  │ git-venv │ —         │ v0.25.0                  │ optional │
│ tt-model             │ uv-tool  │ —         │ a7aa9cefdd3086b091b…     │ install  │
│ tt-smi               │ uv-tool  │ —         │ 6.6.0                    │ install  │
│ tt-studio            │ git-venv │ —         │ v2.12.0                  │ optional │
│ system-stack         │ system   │ —         │ hugepages 1.4.1, kmd     │ converge │
│                      │          │           │ 2.11.0, sfpi 7.71.0,     │          │
│                      │          │           │ firmware 19.15.0 (via    │          │
│                      │          │           │ install.sh 3.6.2)        │          │
└──────────────────────┴──────────┴───────────┴──────────────────────────┴──────────┘
```

Components marked `optional` are fetched automatically the first time a command needs them, so `tt update` leaves them alone unless you pass `--include-lazy`.

Now run the update:

```
tt update
```

`tt update` works in three phases:

1. **Checks.** It fetches the golden version manifest and verifies its checksum.
2. **Tools.** It installs TT-SMI, TT-Flash, and the model manager into isolated environments under `~/.local/share/tenstorrent/`, where they cannot conflict with each other or with your own Python environment.
3. **System.** It downloads the matching tt-installer release and runs it non-interactively. This phase asks for your `sudo` password and installs the base packages, the Tenstorrent package repositories, TT-KMD, HugePages, firmware, and Docker if no container runtime is present. It also installs a `tt-metalium` command that opens the TT-Metalium™ development container, for when you want to go deeper later.

Required

Using sudo is required, so you must enter your user's password when prompted.

`tt update` also asks you to confirm before it flashes firmware, because flashing resets the devices. On a fresh machine it cannot read the current firmware yet, so the prompt is `Continue with tt update? [y/N]` with a warning that continuing may flash and reset your devices. Stop any running model workloads first, then answer **y**. On a machine that already has the driver, the prompt only appears when the firmware is older than golden.

On a TT-QuietBox 2 with a fast connection the whole update takes a few minutes. It ends with a summary like `Update complete: 3 tool(s) updated, 2 optional (--include-lazy), system stack converged.`

Bring the CLI itself up to date

The installer that `tt update` runs also installs the `tt` release that was validated with the golden set, which can be older than the one you installed in Step 1. This page documents the latest release, so before you move on, put the CLI back on it:

```
tt self update
```

It prints `tt upgraded: 1.0.1 → 1.2.0` when it had something to do, or `tt 1.2.0 is the newest release.` when you are already current. Keep both halves current over time: `tt update` converges the system stack and tools onto the latest golden set, and `tt self update` upgrades the CLI. `tt` also checks for a newer release once a day and prints a notice when one is available.

Reboot after the first install

`tt update` never reboots your machine for you. If this is the first time the driver was installed on this machine, or HugePages configuration changed, **reboot now** before continuing:

```
sudo reboot
```

If `tt update` installed Docker, it also adds your user to the `docker` group. That takes effect when you log in again, which the reboot takes care of.

Step 3: Verify Your Hardware

After rebooting, confirm the driver is loaded and that every Tenstorrent device in your system is visible:

```
tt device status
```

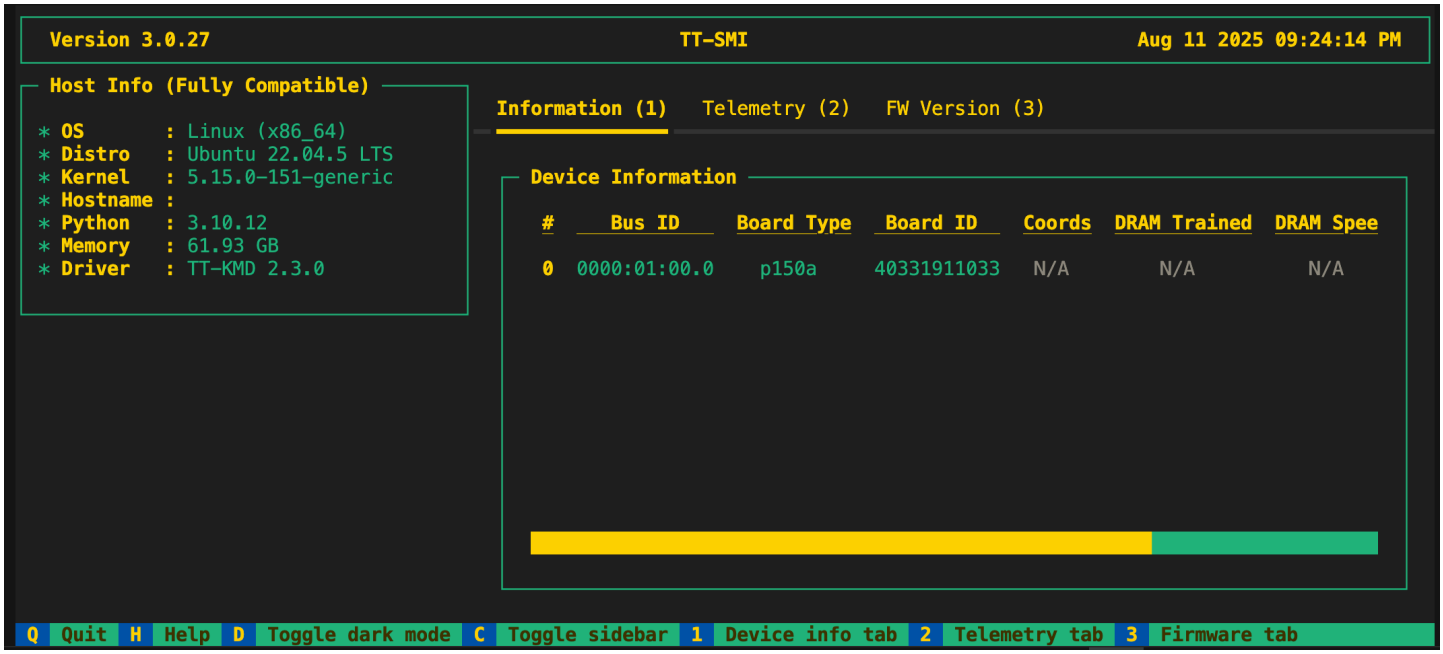
You should see one row per device, with the board type, PCIe bus address, and live telemetry. Here is an example from a TT-QuietBox 2 with four Blackhole p300 cards:

Tenstorrent devices (driver: TT-KMD 2.11.0)

#	Board	Bus ID	Temp	Power	AICLK	Voltage	DRAM
0	p300c	0000:01:00.0	43.9 °C	31.0 W	800.0 MHz	0.71 V	True
1	p300c	0000:02:00.0	44.4 °C	17.0 W	800.0 MHz	0.71 V	True
2	p300c	0000:03:00.0	43.6 °C	16.0 W	800.0 MHz	0.72 V	True
3	p300c	0000:04:00.0	45.1 °C	16.0 W	800.0 MHz	0.72 V	True

The number of rows should match the number of Tenstorrent devices installed in your system. For more detail on a device, such as its firmware versions and board ID, run `tt device info`. For a live, interactive view, run `tt smi`, which hands your terminal to the TT-SMI interface. Press **Q** to leave it.

Here is an example of the TT-SMI interface for a system containing a single p150a device:



Important

If the number of listed devices does not match what you expect, see Troubleshooting below. If the problem persists, contact support and we will assist you.

Congratulations, your system software is installed and your hardware is ready to use.

Step 4: See Which Models You Can Run

`tt` knows your hardware, so it can tell you which models are validated to run on it. List them:

```
tt model list
```

The list is filtered to the device configuration `tt` detected. The rows below are an example only, trimmed from a TT-QuietBox 2 listing, which `tt` detects as a `p300x2` configuration, to show the different ways a model can appear in the list:

```
  Models (tt catalog + community) for p300x2 (detected — `tt model list --all` for every
                                  device/bundle)
┌──────────────────────────┬──────────────────────┬────────┬──────────────────┬─────────┐
│ name                     │ source               │ engine │ serving profiles │ weights │
├──────────────────────────┼──────────────────────┼────────┼──────────────────┼─────────┤
│ FLUX.1-dev               │ tt-inference-server  │ media  │ p300, p300x2     │ —       │
│ Llama-3.3-70B-Instruct   │ tt-inference-server  │ vLLM   │ p150x4, p300x2   │ —       │
│ Qwen3-32B                │ tt-inference-server  │ vLLM   │ p150x4, p300x2   │ —       │
│ Qwen3.5-9B               │ tt-studio            │ vLLM   │ p150             │ —       │
│ Qwen3.8-27B              │ tt-inference-server  │ vLLM   │ p150x4, p300x2   │ —       │
│ tt-hous/qwen3.8-flash-next │ HuggingFace        │ vLLM   │ p300x2           │ —       │
│ whisper-large-v3         │ tt-inference-server  │ media  │ p150             │ —       │
└──────────────────────────┴──────────────────────┴────────┴──────────────────┴─────────┘
```

The columns tell you:

- **source**: where the model comes from. `tt-inference-server` and `tt-studio` rows are the **released catalog**, the models Tenstorrent ships and tests. `HuggingFace` rows are **community bundles** that anyone can publish to the Hugging Face Hub with the Tenstorrent model manager. Community bundles are not tested or maintained by Tenstorrent, and are served by their full `namespace/name` id. `local` marks a bundle already installed on this machine.
- **engine**: how the model is served. `vLLM` models expose an OpenAI-compatible chat API, `media` models expose image, video, or speech endpoints, and `forge` models run through the TT-Forge™ compiler.
- **serving profiles**: the board or mesh configurations the model has a serving configuration for. Your detected configuration is always among them.
- **weights**: whether the model's weights are already downloaded on this machine, and how much space they take.

Useful ways to narrow the list, and to learn more about one model:

```
tt model list --catalog              # only the released catalog
tt model list --type llm             # only large language models
tt model list --all                  # every model on every device, not just this machine's
tt model info Qwen3.8-27B            # requirements, validation status per board, and how to
serve it
```

`tt model info` is the place to check before you serve a model. It reports the minimum disk space and host RAM the model needs, the serving paths it supports, and its validation status on each board: `COMPLETE` models are fully validated, `FUNCTIONAL` models run and produce correct output but may not yet reach target performance, and `EXPERIMENTAL` models are early.

Tip

To find community bundles for a specific model, search the Hub with `tt model search <query>`, for example `tt model search gpt-oss`. The community models guide explains how bundles are packaged, published, and pulled.

Log In to Hugging Face

Catalog models download their weights from the Hugging Face Hub, so serving needs a Hugging Face account and login, even for freely available models. Some models are also gated, which means you must visit the model's page on the Hub and accept its license before you can download it. Meta's Llama and Google's Gemma models are gated; Qwen models are not.

1. Create a Hugging Face account at huggingface.co/join if you do not have one.
2. For a gated model, open its page on the Hub and accept the license agreement. Approval can take a few minutes. The model's `hf_repo` line in `tt model info` tells you which page that is.
3. Log in from your machine so that `tt` can download on your behalf:

```
tt model login
```

`tt model login` offers two ways to log in. **Log in with your browser** shows a URL and a short code: open the URL on any device, enter the code, and approve the request. **Paste an access token** takes a read token you create at huggingface.co/settings/tokens. Either way, the login is saved in Hugging

Face's own token store, so it is also picked up by any other Hugging Face tooling you use, and `tt serve --dry-run` reports it as `HF token: from the Hugging Face login store`.

Step 5: Serve a Model

Serving a model gives you an OpenAI-compatible API endpoint running on your Tenstorrent hardware. Pick an LLM from your `tt model list` output. This guide uses `Qwen3.8-27B`, which is fully validated on the TT-QuietBox 2 and other four-chip Blackhole systems, and needs no license agreement on the Hub. On other hardware pick an LLM from your own list, for example `Qwen3-32B` on a Wormhole TT-LoudBox or `Qwen3-8B` on a single n150 or n300. Check what the model needs first:

```
tt model info Qwen3.8-27B
```

Qwen3.8-27B needs 74 GB of free disk space and 67.5 GB of host memory, and `tt model info` reports it as `COMPLETE` on the p300x2 configuration of a TT-QuietBox 2. Serve it:

```
tt serve Qwen3.8-27B
```

`tt serve` picks the right serving path for the model, installs TT-Inference-Server on first use, pulls the model's container image, downloads the weights, opens the device, loads the weights, and warms the model up. It shows those stages as a live checklist and returns only once the endpoint actually answers requests:

```
  ✓ host ready  tt-inference-server 0.25.0
  ✓ weights ready  Qwen/Qwen3.8-27B  6m 53s
  ✓ image ready  vllm-tt-metal-src-release-ubuntu-22.04-amd64:0.25.0  1m 37s
  ✓ container started  tt-inference-server-d02c18d2  3.8s
  ✓ engine started  vLLM 0.26.0  17.8s
  ✓ Tenstorrent device opened  4 chips · mesh (1, 4)  7.0s
  ✓ KV cache configured  1,051,104 tokens  3m 28s
  ✓ model warmed up  209s of warmup
  ✓ API server started  1.8s
  ✓ endpoint answering
✓ Phase 3/3 · Start  15m 26s
┌─ Qwen3.8-27B ready ──────────────────────────────────────────────────────────┐
│                                                                              │
│  endpoint  http://127.0.0.1:20000/v1                                         │
│  models    curl http://127.0.0.1:20000/v1/models                             │
│  chat      tt launch                                                         │
│  Ready in 15m 26s · via tt-inference-server                                  │
│  Logs · tt model logs Qwen3.8-27B --follow                                   │
│  Stop · tt model stop Qwen3.8-27B                                            │
└──────────────────────────────────────────────────────────────────────────────┘
```

Note

The first serve of a model takes a while: the container image (about 18 GB) and the weights (56 GB for Qwen3.8-27B) are downloaded, then the model is compiled and warmed up on your hardware. The run above is a first serve on a TT-QuietBox 2 and took 15 minutes, 7 of them downloading weights. Everything it downloaded and compiled is cached, so later serves of the same model are faster: about seven minutes for Qwen3.8-27B on a TT-QuietBox 2, most of it sizing the KV cache and warming up. `tt model pull Qwen3.8-27B` downloads the weights ahead of time.

To watch the raw server log while you wait, run `tt model logs Qwen3.8-27B --follow` in a second terminal. Pressing Ctrl-C in the `tt serve` terminal stops watching, not the server.

When the ready box appears, the model is being served on port 20000. Send it a first request:

```
tt model curl "In one sentence, what is Tenstorrent?" --max-tokens 2048
```

`tt model curl` asks the server which model it is serving, sends your prompt, and prints the server's JSON chat completion. Qwen3.8-27B is a reasoning model: it thinks first, in a separate `reasoning` field of the reply, and then answers in `content`. Its default budget of 64 tokens is used up by the

thinking, which is why the example raises it with `--max-tokens`; any option `tt model curl` does not recognize goes straight into the request body. The one-sentence answer arrives in a few seconds. An open question such as "What can you tell me about Tenstorrent?" takes a couple of minutes, because the model reasons at length before it writes. Add `--print` to see the equivalent `curl` command instead of sending it. The endpoint speaks the OpenAI chat completions API, so any OpenAI-compatible client can use it: point the client at `http://localhost:20000/v1` with any value for the API key. Use `--port` on `tt serve` to choose a different port.

A few commands to manage what is running:

```
tt model ps              # what is being served, on which port, and whether it is healthy
tt model logs Qwen3.8-27B  # the server's log
tt model stop Qwen3.8-27B  # stop the server
```

Tip

`tt serve <model> --dry-run` shows the backend, device configuration, container image, and port that would be used without starting anything. It is a quick way to confirm `tt` detected your hardware the way you expect.

Step 6: Connect an App with `tt launch`

An API endpoint is the foundation. `tt launch` connects a ready-made application to it, so you can start using the model right away. See which clients are available on your machine:

```
tt launch list
```

```
                                                tt launch
┌──────────────┬──────────────┬──────────────────┬──────────────────┬──────────────────────────────────────────────
──────┐
│ client       │ kind         │ model            │ status           │ configures
│
├──────────────┼──────────────┼──────────────────┼──────────────────┼──────────────────────────────────────────────
──────┤
│ opencode     │ terminal     │ tool calling     │ needs opencode   │
~/.config/opencode/opencode.json     │
│ pi           │ terminal     │ tool calling     │ needs pi         │ ~/.pi/agent/models.json
│
│ aider        │ terminal     │ tool calling     │ needs aider      │ environment (this run only)
│
│ qwencode     │ terminal     │ tool calling     │ needs qwen       │ command-line arguments (this
run only) │
│ hermes       │ terminal     │ tool calling     │ needs hermes     │ environment (this run only)
│
│ openwebui    │ web service  │ any chat model   │ ready            │ container tt-open-webui
│
│ anythingllm  │ web service  │ any chat model   │ ready            │ container tt-anythingllm
│
└──────────────┴──────────────┴──────────────────┴──────────────────┴──────────────────────────────────────────────
──────┘
```

There are two kinds of client:

- **Web UIs** such as Open WebUI and AnythingLLM. `tt` pulls and runs these for you in a container, after asking once. They show as `ready` as soon as Docker is available.
- **Terminal coding agents** such as OpenCode, Aider, pi, Qwen Code, and Hermes Agent. You install the agent yourself, and `tt` writes the small configuration it needs to reach your model. These need a model that supports tool calling, which `tt` checks for you.

With the model from Step 5 still being served, launch a chat interface in your browser:

```
tt launch openwebui
```

`tt` asks the server which model it is serving, then asks you to confirm before it pulls the image: `Pull ghcr.io/open-webui/open-webui:main (a few GB) and run it as tt-open-webui on port 3000. Continue? [y/N]`. It starts the container, prints a summary of what it connected, and ends with `tt-open-webui is ready at http://localhost:3000`. Open that URL in your browser. No sign-in is needed: pick the model and start chatting with it running on your Tenstorrent hardware.

When you are done:

```
tt launch stop openwebui   # stop the web UI; its data is kept
tt model stop Qwen3.8-27B  # stop the model server
```

The full list of clients, what each one needs installed, and the options for connecting to a server on another machine are in the client launchers guide.

Prefer a point-and-click experience? Use TT-Studio

TT-Studio is a web application that deploys models and gives you a browser interface for them in one place. It is a serving path rather than a `tt launch` client, so you start it with `tt serve`. TT-Studio takes the devices and port 3000 for itself, so stop the web UI and the model server from the steps above first, and reset the devices, because a model server that was stopped by `tt model stop` leaves them in a state TT-Studio's deploy cannot initialize:

```
tt launch stop openwebui
tt model stop Qwen3.8-27B
tt device reset
tt serve Qwen3.8-27B --studio
```

On first use `tt` fetches the latest TT-Studio release. TT-Studio asks you to accept its terms and offers to add a `tt-studio` shortcut to your shell, which you can decline. It then brings up its services and prints a `TT Studio is ready` box with its web UI at `http://localhost:3000`, deploys the model, and prints a `Qwen3.8-27B is ready` box with the endpoint, again on port 20000, and the page that manages the deployment, `http://localhost:3000/models-deployed`. With the images already downloaded this takes a few minutes. TT-Studio requires Docker.

Stopping works as before. `tt model stop Qwen3.8-27B` stops the model through TT-Studio, resets the devices, and takes TT-Studio's services down with it, keeping its settings, downloaded weights, and chat history for next time.

Troubleshooting

`tt device status` reports no devices

If `tt device status` exits with a message that the driver may not be installed or no card is seated:

1. If you have not rebooted since `tt update` installed the driver, reboot now.
2. Run `lspci -d 1e52` to check whether the card has enumerated on the PCIe bus. If it returns nothing, the host cannot see the card at all.
3. For Blackhole PCIe products such as p100 and p150, verify the card is being supplied power by checking that the fan spins and the green power LED illuminates when the system boots. If the light is on, try resetting the card with `tt device reset`.
4. Confirm the BIOS setting described in BIOS Requirements.

`tt serve` stops before starting, asking for a Hugging Face token

Catalog models require a Hugging Face login to download their weights. Run `tt model login` as described in Step 4.

`tt serve` fails with "Access to model … is restricted"

The model is gated and your Hugging Face account has not accepted its license yet, or the login on this machine is for a different account. Accept the license on the model's Hub page, check `tt model login`, and serve again. If the error persists at the `starting the engine` step after both, serve a model that is not gated, such as `Qwen3.8-27B`. `tt model logs <model>` shows the server's own output for any failed start.

A serve fails with "Try resetting the board"

A model server that was stopped or crashed can leave the devices in a state the next server cannot use. The server log then ends with a `TT_THROW` message that asks you to reset the board. Reset every device and serve again:

```
tt device reset
```

`tt device reset` asks for confirmation because it interrupts any running workload. `tt device status` should show the devices back at their idle clock afterwards.

`tt launch` says port 3000 is already in use

Another service holds port 3000, for example TT-Studio or a development server of your own. The web UI container keeps the port it was created with, so `tt launch` prints the exact commands to recreate it on another port: `docker rm tt-open-webui && tt launch openwebui --web-port 3001`. The container's chat history lives in a separate volume and survives the recreate.

Docker commands are refused with a permission error

`tt` talks to Docker as your user, never through `sudo`. If `tt update` just installed Docker, log out and back in (or reboot) so that your membership in the `docker` group takes effect. You can check with `docker run hello-world`.

Getting help

`tt report bundle` collects a redacted support bundle with your environment details, a TT-SMI snapshot, and recent logs, and opens a prefilled email to Tenstorrent support with it attached. `tt report issue` opens a prefilled GitHub issue on the tt-cli repository.

Alternative Installation Methods

`tt update` is the recommended way to install the system software, and it runs tt-installer for you with the tested versions. If you prefer to drive tt-installer interactively yourself and choose each of its options, you can run it directly:

```
/bin/bash -c "$(curl -fsSL https://tenstorrent.ai/install.sh)"
```

For advanced users or developers who prefer to install and configure each component by hand, refer to the manual software installation guide.

Where to Go Next

You now have the full stack installed and a model serving from the CLI. From here, branch off into the part of the stack that matches what you want to build:

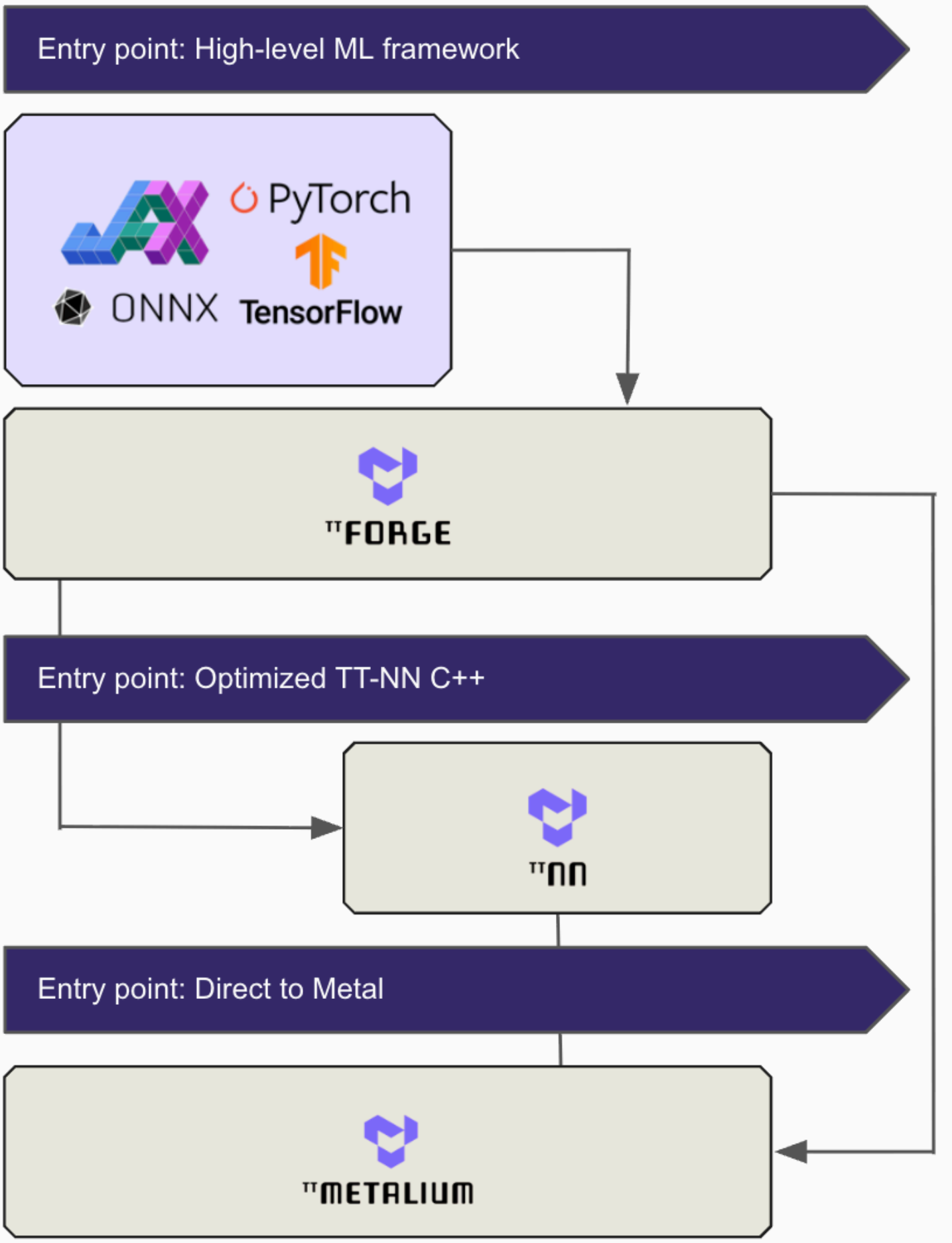
- **Deploy more kinds of model** by following the Deploy Models guide: choosing a model for your hardware, serving LLMs, diffusion, speech, and vision models from the CLI, and deploying the same models through TT-Studio's web interface.
- **Monitor your hardware** with `tt smi` for a status snapshot, or TT-Toplike for a live view of power, temperature, and core activity while a model serves requests.
- **Use a web interface** with TT-Studio, started through `tt serve <model> --studio`.
- **Understand the software stack**, including how TT-Forge™, TT-NN™, and TT-Metalium™ relate to each other, in the software stack overview.
- **Compile your own PyTorch, JAX, or ONNX models** with TT-Forge™.
- **Run and build models in Python** with TT-NN™, or **write custom kernels** with TT-Metalium™. `tt update` already installed a `tt-metalium` command that opens a shell inside the TT-Metalium container, with your home directory mounted and TT-NN ready to import; the image, about 4.5 GB, is pulled the first time you run it. Start with the Metalium guide to learn about the architecture.
- **Learn by building** with the hands-on lessons in the TT VS Code Toolkit.

Need Additional Support?

If you encounter any issues, or have a question that isn't covered in the documentation, please raise a support request. Our team will review your request and provide assistance.

Overview

This document provides developers and researchers with an overview of the main components of the Tenstorrent software stack: TT-Metalium™, TT-NN™, and TT-Forge™. You will learn about each tool's purpose, use cases, and where to find its source code.


Entry point: High-level ML framework
PyTorch
ONNX
TensorFlow
TTFORGE
Entry point: Optimized TT-NN C++
TTNN
Entry point: Direct to Metal
TTMETALIUM

TT-Forge™: The MLIR-Based Compiler

TT-Forge™ is Tenstorrent's Multi-Level Intermediate Representation (MLIR)-based compiler. It bridges high-level machine learning frameworks with the Tenstorrent software stack.

Interact with TT-Forge™ to compile models from frameworks such as PyTorch, JAX, and TensorFlow for execution on Tenstorrent hardware. It offers an automated, general path to run many types of model architectures without requiring custom kernel development. TT-Forge™ integrates with and lowers to TT-Metalium for hardware execution.

The main project is located in the tt-forge GitHub repository.

TT-NN™: A Python & C++ Neural Network OP library

TT-NN™ is a library of neural network operations that provides a user-friendly interface for running models on Tenstorrent hardware. It is designed to be intuitive for developers familiar with PyTorch.

Interact with TT-NN™ to run AI models using a familiar, high-level Python API without managing the complexities of the underlying hardware. TT-NN™ builds upon TT-Metalium™ and provides a stable set of pre-packaged, optimized operations. It is also available with a C++ API.

The TT-NN™ library is part of the tt-metal GitHub repository.

TT-Metalium™: Programming Tenstorrent Hardware

TT-Metalium™ is the low-level, open-source software development kit (SDK) that provides developers direct access to Tenstorrent hardware. It is a bare-metal programming environment designed for users who must write custom C++ kernels for machine learning or other high-performance computing workloads.

Interact with TT-Metalium™ when you require complete control over the hardware to optimize code for performance, explicitly manage memory, or implement novel operations not found in standard libraries. This environment exposes the RISC-V processors, the Network-on-Chip (NoC), and the matrix and vector engines within each Tensix core.

The main project is available in the tt-metal GitHub repository.

Running Models on Hardware

Once you have the software stack installed, these are the primary entry points for running models:

- **TT-Inference-Server** — manages Docker containers, model downloads, and serving configuration, and provides an OpenAI-compatible API endpoint. It is the authoritative source for which models are validated on each hardware generation.
- **TT-Forge Models** — the catalog of models that have been validated against TT-Forge™. The starting point when compiling a PyTorch, ONNX, or JAX model against the stack.
- **TT-Studio** — a web interface over TT-Inference-Server for users who prefer a point-and-click model deployment flow.

For monitoring your hardware while models run, TT-Toplike reads power, temperature, and core activity in real time from the chips.

Exploring our Developer Hub

To continue learning, visit and explore our Developer Hub. There you will find in-depth articles, and information on our software bounty program. You can also find tutorials and technical overviews on our YouTube channel.

Engaging with our Community

For real-time discussions, community Q&A, and to share your latest projects, we invite you to join our Discord channel. It's a great place to engage directly with other developers and members of the Tenstorrent community.

As always, if you have any questions, please raise a support request with our team. We're here to help, and are excited to see what you build.

PREVIOUS NEXT